UNIDAD 6. Programación Orientada a Objetos Avanzada

Índice

1. Relaciones entre Clases y Herencia 2
   1.1 Relaciones básicas: La Composición frente a la Herencia 2
   1.2 Concepto de Herencia: Superclases y Subclases 4
   1.3 Tipos de Herencia: Simple vs Múltiple 5
   1.4 Constructores y Herencia 7
2. Modificadores y Diseño de Clases 9
   2.1. Revisión de Modificadores: Acceso (protected) y comportamiento (static) 9
   2.2. Sobreescritura de métodos (Overriding) 11
   2.3. Restricciones: Clases y métodos `final` (Inmutabilidad y prohibición de herencia) 14
   2.4. Abstracción: Clases y métodos `abstract` (Plantillas incompletas) 16
3. Interfaces 18
   3.1. Concepto y Definición: Contratos de comportamiento 18
   3.2. Interfaces vs Clases Abstractas: Cuándo usar cada una y diferencias clave 20
4. Polimorfismo 23
   4.1. Concepto General: Una interfaz, múltiples formas 23
   4.2. Polimorfismo en Compilación: Sobrecarga de métodos (Overloading) 25
   4.3. Polimorfismo en Ejecución: Ligadura dinámica (Dynamic Binding) 27
   4.4. Comprobación de tipos: Estática (compilador) vs Dinámica (`instanceof`) 30
5. Tipado Avanzado y Genéricos 32
   5.1. Conversiones de Objetos (Casting) 32
   5.2. Genéricos: Introducción a las clases y tipos parametrizados (`<T>`) 34
Resumen 36

1. Relaciones entre Clases y Herencia

1.1 Relaciones básicas: La Composición frente a la Herencia

Para decidir cómo relacionar dos clases (digamos, Clase A y Clase B), debes hacerte una pregunta lingüística muy simple sobre la realidad que estás programando.

A. Herencia: La relación "Es-Un" (Is-A)

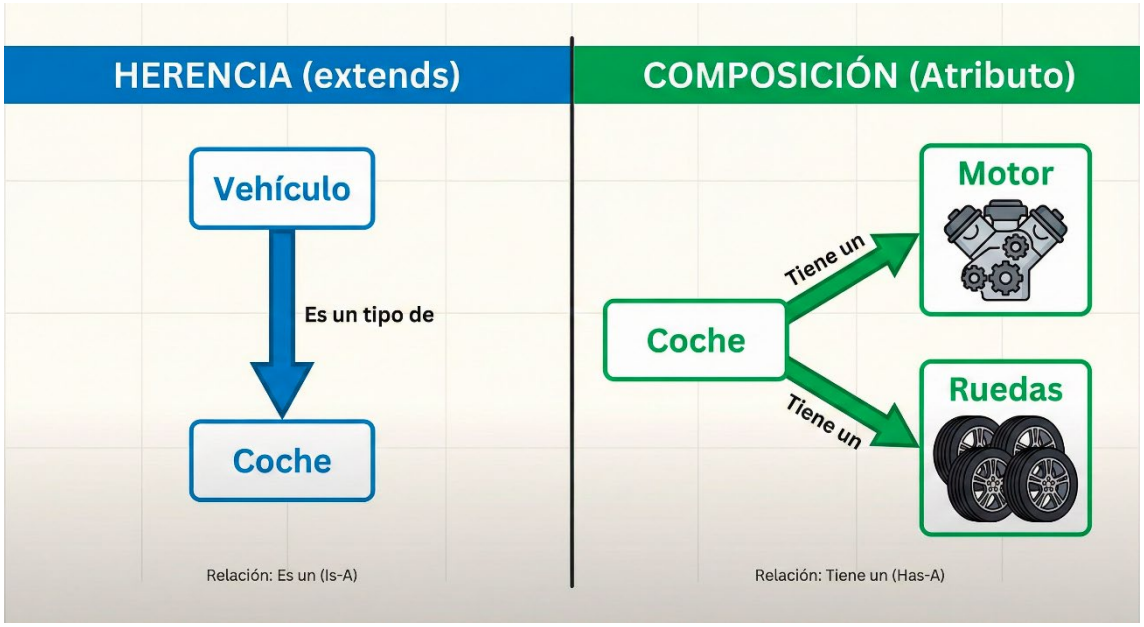
La herencia representa **especialización**. Usamos herencia cuando queremos crear una versión más específica de algo que ya existe.

- **La Pregunta:** ¿B **ES UN** tipo de A?
  - ¿Un `Gato` **ES UN** `Animal`? -> **SÍ**. (Herencia correcta).
  - ¿Un `Coche` **ES UNA** `Rueda`? -> **NO**. (Herencia incorrecta).
- **Mecánica:** La clase hija absorbe (hereda) automáticamente todos los atributos y métodos del padre.
- **Palabra clave:** `extends`.

B. Composición: La relación "Tiene-Un" (Has-A)

La composición representa **ensamblaje**. Usamos composición cuando construimos un objeto complejo uniendo piezas más pequeñas. No es una relación de padre-hijo, sino de **Todo-Parte**.

- **La Pregunta:** ¿A **TIENE UN** B dentro?
  - ¿Un `Coche` **TIENE UN** `Motor`? -> **SÍ**. (Composición correcta).
  - ¿Una `Casa` **TIENE UNA** `Cocina`? -> **SÍ**.
- **Mecánica:** La clase principal declara una **variable de instancia** cuyo tipo es la otra clase.
- **Palabra clave:** Ninguna especial. Simplemente usas `new`.



C. Ejemplo Comparativo: El Coche

Veamos cómo se implementan ambas relaciones en el mismo objeto.
Java

```
// --- HERENCIA (Relación ES-UN) ---
// Un Coche ES UN Vehículo.
// Hereda la capacidad de moverse sin escribir código nuevo.
public class Coche extends Vehiculo {

    // --- COMPOSICIÓN (Relación TIENE-UN) ---
    // Un Coche NO es un Motor, pero TIENE UN Motor dentro para
funcionar.
    private Motor motor;       // Composición
    private Rueda[] ruedas;    // Composición (tiene varias ruedas)

    public Coche() {
        // En el constructor, creamos las piezas (el ensamblaje)
        this.motor = new Motor("V8");
        this.ruedas = new Rueda[4];
    }
}
```

D. Tabla de Decisión: ¿Cuál elijo?

Un error muy común es abusar de la Herencia. Aquí tienes las reglas para elegir bien:

Característica	Herencia (`extends`)	Composición (Atributo)
Relación	**Es un** (Vertical / Jerárquica)	**Tiene un** (Horizontal / Ensamblaje)
Acoplamiento	**Muy Alto (Rígido)**. Si cambias la clase Padre, rompes la Hija automáticamente.	**Bajo (Flexible)**. Puedes cambiar el motor del coche sin rediseñar todo el coche.
Reutilización	Reutilización de **Caja Blanca** (Ves las tripas del padre).	Reutilización de **Caja Negra** (Solo usas la interfaz pública del componente).
Cardinalidad	1 a 1 (Un coche solo puede ser un vehículo).	1 a N (Un coche puede tener 4 ruedas).

Principio de Diseño de Software: "Favorece la Composición sobre la Herencia".

La herencia es muy potente, pero te "casa" de por vida con la clase padre. La composición te permite cambiar piezas (objetos) en tiempo de ejecución, lo que hace tu código mucho más flexible. Úsala siempre que puedas, y usa Herencia solo cuando la relación "Es-Un" sea evidente e inmutable.

1.2 Concepto de Herencia: Superclases y Subclases

La herencia nos permite definir una clase general y luego definir clases más especializadas que añaden o modifican funcionalidades, sin tener que reescribir el código que ya funciona.

A. La Terminología: ¿Quién es quién?

En esta relación siempre intervienen dos actores:

1. **Superclase (Clase Padre / Clase Base):** Es la clase original, la que contiene el código genérico y común.

   - Ejemplo: `Vehiculo`, `Animal`, `Persona`.

2. **Subclase (Clase Hija / Clase Derivada):** Es la clase nueva que hereda de la superclase.

   - **Recibe:** Todos los atributos y métodos `public` y `protected` del padre.
   - **Añade:** Sus propios atributos y métodos exclusivos.
   - Ejemplo: `Coche` (hija de Vehiculo), `Gato` (hija de Animal).

B. Sintaxis: La palabra clave `extends`

Para establecer esta relación en Java, usamos la palabra clave **`extends`** en la declaración de la clase hija. Se lee como "amplía" o "extiende" la funcionalidad del padre.

Java
```java
// 1. SUPERCLASE (Lo genérico)
public class Dispositivo {
    public String marca;
    public boolean encendido = false;

    public void encender() {
        this.encendido = true;
        System.out.println("Dispositivo encendido");
    }
}

// 2. SUBCLASE (Lo específico)
// Al poner 'extends Dispositivo', Movil ya tiene 'marca' y
'encender()'
public class Movil extends Dispositivo {
    public int bateria; // Atributo exclusivo de Movil

    public void llamar() { // Método exclusivo de Movil
        System.out.println("Llamando...");
    }
}
```

Uso en el Main: Fíjate cómo el objeto `m` (tipo Movil) tiene acceso a cosas suyas y cosas de su padre.

Java
```java
Movil m = new Movil();
```

```
m.marca = "Samsung"; // Heredado de Dispositivo
m.bateria = 90;      // Propio de Movil

m.encender();        // Heredado de Dispositivo
m.llamar();          // Propio de Movil
```

C. La Jerarquía de Clases (El Árbol Genealógico)

La herencia no se limita a dos niveles (Padre -> Hijo). Un hijo puede ser padre de otro, creando una **Jerarquía de Herencia** multinivel.

- `Animal` (Abuelo)
  - `Mamifero` (Padre - Hereda de Animal)
    - `Perro` (Hijo - Hereda de Mamifero y, por tanto, de Animal)

Regla de Tránsito: Si `Perro` hereda de `Mamifero`, y `Mamifero` hereda de `Animal`, entonces `Perro` tiene automáticamente todo lo que tenga `Animal`.

D. `java.lang.Object`: La madre de todos

En Java, existe una regla oculta fundamental: **Toda clase que creas hereda implícitamente de la clase `Object`.**

Si tú escribes `public class Coche`, Java interpreta `public class Coche extends Object`.

- Esto significa que **TODAS** las clases en Java (arrays y Strings incluidos) forman una única jerarquía gigante con `Object` en la cúspide.
- Por eso cualquier objeto tiene métodos como `.toString()`, `.equals()` o `.hashCode();` porque los heredan de su "tatarabuelo" `Object`.

1.3 Tipos de Herencia: Simple vs Múltiple

A. Herencia Simple (El Modelo Java)

Java sigue un modelo de **Herencia Simple**. Esto significa que una clase solo puede tener **una única superclase** directa.

- **Regla:** Después de la palabra `extends`, solo puedes escribir **un nombre**.
  - `public class Coche extends Vehiculo` ✅
  - `public class Coche extends Vehiculo, Motor` ❌ **ERROR DE COMPILACIÓN**
- **Estructura:** Esto genera una estructura de árbol limpia. Si dibujas las relaciones, nunca verás líneas que se crucen o se unan desde arriba.

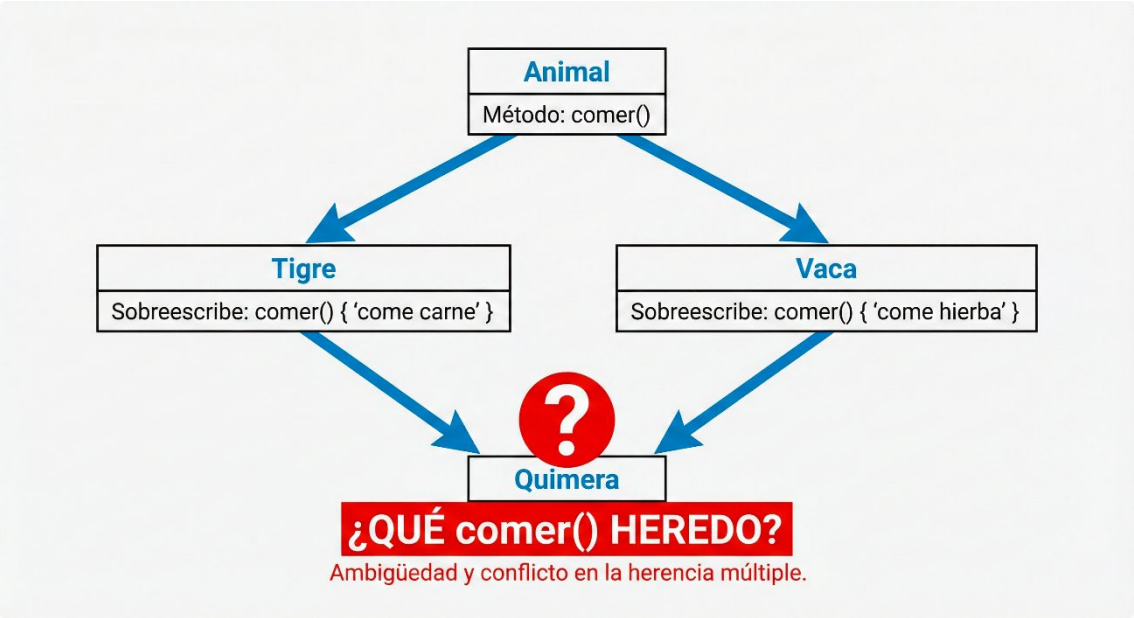
B. El problema de la Herencia Múltiple (¿Por qué no?)

Lenguajes como C++ o Python permiten herencia múltiple. ¿Por qué Java la prohibió? Para evitar el **Problema del Diamante**.

Imagina este escenario de caos:

1. Clase `Animal` tiene el método `comer()` ("Come comida").
2. Clase `Tigre` (hija de Animal) sobreescribe `comer()` ("Come carne").
3. Clase `Vaca` (hija de Animal) sobreescribe `comer()` ("Come hierba").
4. Creamos una Clase `Quimera` que hereda de `Tigre` y `Vaca` a la vez.

La pregunta mortal: Cuando hacemos `quimera.comer()`, ¿qué código se ejecuta? ¿El del Tigre o el de la Vaca? El compilador no sabría qué hacer. Para evitar esta ambigüedad y complejidad, Java cortó por lo sano: prohibido.



C. Herencia Múltiple "Simulada" (Interfaces)

Aunque no podemos heredar código de dos sitios, a menudo necesitamos que un objeto sea "varias cosas a la vez" (ej: que un `Hidroavion` sea `Avion` y `Barco`).

Java soluciona esto separando "SER" de "COMPORTARSE":

1. **Herencia de Clases (`extends`):** Solo una. (Define qué **ES** el objeto).
2. **Implementación de Interfaces (`implements`):** Infinitas. (Define qué **ROLES** cumple o qué **HACE**).

Veremos las Interfaces a fondo más adelante, pero quédate con esta sintaxis que es el "truco" de Java para simular herencia múltiple:

Java
```
// Solo un padre biológico (extends)
// Pero muchos roles/títulos (implements)
public class Hidroavion extends Vehiculo implements Navegable,
Volable {

    // Hereda motor y chasis de Vehiculo.
```

```
    // Y está OBLIGADO a definir cómo navega y cómo vuela
    // para cumplir con las interfaces.
}
```

Resumen:

- En Java, la herencia de código (clases) es **Simple**.
- La herencia de tipos (interfaces) es **Múltiple**.

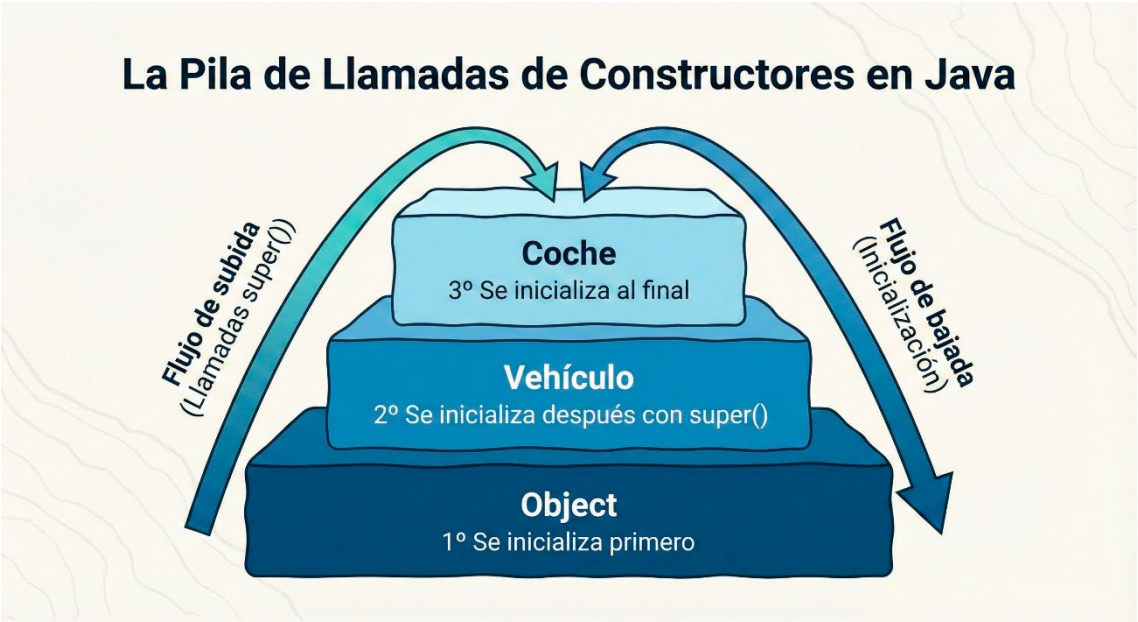
1.4 Constructores y Herencia

A. El Orden de Inicialización

Cuando haces `new Coche()`, podrías pensar que solo se ejecuta el constructor de `Coche`. **Falso**.

Java sigue una regla estricta: "Los padres siempre se inicializan antes que los hijos". El orden de ejecución real es desde la clase más alta (`Object`) bajando hasta tu clase.

1. Se ejecuta el constructor de `Object`.
2. Se ejecuta el constructor de `Vehiculo` (Padre).
3. Se ejecuta el constructor de `Coche` (Hijo).



B. La palabra clave `super()`

Para cumplir la regla anterior, Java utiliza la instrucción especial super(). super() es una llamada explícita al constructor de la clase padre.

Las 2 Reglas de Oro de `super()`:

1. **Ubicación:** Si lo escribes, debe ser obligatoriamente la **PRIMERA línea** del constructor del hijo.
2. **Automatismo:** Si tú **NO** escribes nada, Java inserta automáticamente una llamada oculta a `super()` (sin argumentos) en la primera línea.

Java
```
public class Hijo extends Padre {
    public Hijo() {
        // Aunque no lo veas, Java escribe aquí: super();
        System.out.println("Soy el hijo");
    }
}
```

C. El problema de los Constructores con Parámetros

Aquí es donde surgen los errores de compilación. Recordemos la regla de la Unidad 4: *Si escribes un constructor con parámetros en el Padre, el constructor vacío por defecto desaparece.* Si el Padre **solo** tiene constructores con parámetros, el `super()` automático (vacío) que Java intenta poner en el Hijo **fallará**, porque no existe un constructor vacío en el Padre al que llamar.

Solución: El Hijo está **OBLIGADO** a llamar manualmente a `super(argumentos)` pasándole los datos que el Padre necesita.

Ejemplo de Encadenamiento Correcto:
Java
```
// CLASE PADRE
public class Persona {
    String nombre;

    // El padre EXIGE un nombre para nacer. No hay constructor
vacío.
    public Persona(String nombre) {
        this.nombre = nombre;
    }
}

// CLASE HIJA
public class Empleado extends Persona {
    double sueldo;

    // El hijo recibe nombre y sueldo
    public Empleado(String nombre, double sueldo) {

        // 1. PRIMERO: Satisfacemos al padre pasándole el nombre
        super(nombre);

        // 2. DESPUÉS: Inicializamos lo nuestro
        this.sueldo = sueldo;
    }
}
```

Analogía: Imagina que `Persona` es un trámite que requiere el DNI. Si creas un trámite derivado `Empleado`, no puedes "saltarte" el requisito del DNI. El constructor de `Empleado` debe pedir el DNI y pasárselo inmediatamente al constructor de `Persona`.

D. Diferencia entre `this()` y `super()`

A veces verás `this()` y a veces `super()`. Ambos sirven para llamar a constructores, pero con direcciones distintas:

- **`super()`**: Llama al constructor del **PADRE** (sube un nivel en la jerarquía).
- **`this()`**: Llama a otro constructor de la **MISMA CLASE** (se mueve lateralmente, útil para sobrecarga).

Restricción: Como ambos exigen ser la **primera línea** del código, **nunca** pueden estar juntos en el mismo constructor. Tienes que elegir uno.

2. Modificadores y Diseño de Clases

2.1. Revisión de Modificadores: Acceso (protected) y comportamiento (static)

El diseño de clases no solo trata de qué métodos escribes, sino de **quién** puede verlos y **cuándo** existen en memoria.

A. El modificador de acceso `protected`

Este es, a menudo, el modificador más incomprendido. Se sitúa entre el acceso por defecto (package-private) y el acceso público.

¿Qué permite protected?

Un miembro (variable o método) marcado como protected es visible para:

1. Cualquier clase dentro del **mismo paquete** (igual que *default*).
2. Cualquier **subclase** (hija), sin importar en qué paquete se encuentre.

   Regla de Oro: Piensa en `protected` como el modificador de la **"Herencia"**. Lo usas cuando quieres ocultar algo al mundo exterior, pero necesitas que las clases que extienden tu funcionalidad puedan acceder a ello.

Ejemplo de Diseño: El Patrón "Template Method" Imagina que estás diseñando una clase padre para generar reportes. Quieres que el proceso general sea fijo, pero que los pasos específicos sean personalizables por los hijos.

Java
```java
package com.empresa.reportes;

public class GeneradorReporte {
    // PUBLIC: Cualquiera puede pedir el reporte
    public void generar() {
        abrirArchivo();
        escribirContenido(); // Llama al método protegido
        cerrarArchivo();
    }

    // PROTECTED: Solo las hijas pueden decidir CÓMO escribir el
contenido.
```

```
    // El usuario final no debería llamar a esto directamente.
    protected void escribirContenido() {
        System.out.println("Contenido genérico...");
    }

    // PRIVATE: Nadie más necesita saber cómo se manejan los
archivos internamente
    private void abrirArchivo() { ... }
    private void cerrarArchivo() { ... }
}
```

Ahora, una clase en **otro paquete** puede extender esto:
Java

```
package com.empresa.finanzas;
import com.empresa.reportes.GeneradorReporte;

public class ReporteFinanciero extends GeneradorReporte {
    @Override
    protected void escribirContenido() {
        // Acceso permitido porque es PROTECTED y esto es una
herencia
        System.out.println("Escribiendo datos financieros
confidenciales...");
    }
}
```

B. El modificador de comportamiento `static`

El modificador `static` cambia las reglas del juego respecto a la memoria y la pertenencia. Cuando algo es estático, **pertenece a la clase**, no a una instancia (objeto) individual.

1. Variables Estáticas (`static fields`)

- **Concepto:** Existe una **única copia** de la variable en memoria, compartida por todos los objetos de esa clase.
- **Uso común:** Contadores compartidos, constantes globales, configuraciones generales.

Java

```
public class Usuario {
    // Instancia: Cada usuario tiene su propio nombre
    private String nombre;

    // Static: Solo hay un contador para TODOS los usuarios
    private static int contadorUsuarios = 0;

    public Usuario(String nombre) {
        this.nombre = nombre;
        contadorUsuarios++; // Incrementa la variable compartida
    }
}
```

2. Métodos Estáticos (`static methods`)

- **Concepto:** Son métodos que se pueden ejecutar sin necesidad de crear un objeto (`new Class()`).

- **Restricción Importante:** Un método estático **NO** puede acceder a variables de instancia (no estáticas) ni usar `this`, porque al ejecutarse, es posible que no exista ninguna instancia en memoria.
- **Uso común:** Métodos de utilidad (como `Math.sqrt()`), factories o validadores puros.

Comparativa Rápida:

Característica	Miembros de Instancia (Sin static)	Miembros Estáticos (static)
Pertenencia	Pertenecen a un objeto específico (ej. `usuario1`).	Pertenecen a la clase (ej. `Usuario`).
Memoria	Se crea una copia por cada `new`.	Se crea una sola copia al cargar la clase.
Acceso a `this`	Sí.	**No.**
Invocación	`objeto.metodo()`	`Clase.metodo()`

Impacto en el Diseño

Al diseñar tus clases, se considera:

1. **Encapsulamiento con `protected`:** No hagas todo `public`. Si un método es solo para uso interno de la jerarquía de herencia, usa `protected`. Mantiene tu API limpia para el usuario final.
2. **Estado Global con `static`:** Ten cuidado con las variables estáticas mutables (que cambian). Son esencialmente variables globales y pueden causar errores difíciles de rastrear si muchos objetos las modifican a la vez.
3. **Utility Classes:** Si tienes una clase que es solo una colección de métodos de ayuda y no guarda estado, haz todos sus métodos `static` y su constructor `private` (para evitar que se instancie).

2.2. Sobreescritura de métodos (Overriding)

La sobreescritura (o *overriding*) ocurre cuando una subclase (hija) proporciona una implementación específica de un método que ya está definido en su superclase (padre).

Es la forma en que Java dice: "Heredo esta capacidad de mi padre, pero yo la hago a mi manera".

A. La Anotación `@Override`

Aunque es opcional, usar la anotación `@Override` es una de las mejores prácticas de seguridad en Java. Se coloca justo encima de la firma del método.

¿Para qué sirve? Sirve como un **control de calidad** en tiempo de compilación. Le dice al compilador: "*Oye, mi intención es modificar un método que ya existe en el padre. Si no lo encuentras, avísame con un error*".

Ejemplo sin `@Override` (El peligro de los errores silenciosos):

Java
```
class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {
    // ERROR SUTIL: Escribí mal "hacerSonido" (hacersonido).
    // Java cree que estoy creando un método NUEVO, no cambiando el
anterior.
    public void hacersonido() {
        System.out.println("Guau");
    }
}

// Al ejecutar:
// Perro p = new Perro();
// p.hacerSonido(); -> Imprime "Sonido genérico" (¡No lo que
queríamos!)
```

Ejemplo con `@Override` (La red de seguridad):

Java
```
class Perro extends Animal {
    @Override
    public void hacersonido() { // EL COMPILADOR DARÁ ERROR AQUÍ
        System.out.println("Guau");
    }
}
// Mensaje del compilador: "Method does not override method from
its superclass"
// Nos obliga a corregir el nombre a 'hacerSonido'.
```

B. Modificación del comportamiento heredado

Al sobreescribir, tienes dos estrategias principales: **Reemplazar** totalmente o **Extender** la funcionalidad.

1. Reemplazo Total. Ignoras por completo lo que hacía el padre y defines una lógica nueva.

Java
```
class Transporte {
    public int obtenerVelocidad() {
        return 10;
    }
}

class Avion extends Transporte {
    @Override
    public int obtenerVelocidad() {
        return 900; // Ignoramos el 10, devolvemos 900.
    }
}
```

2. Extensión (Uso de `super`) A veces quieres conservar la lógica del padre y añadir algo más. Aquí usamos la palabra clave `super` para llamar a la versión original del método.

> **Patrón común:** Muy usado en inicializaciones o métodos de guardado donde el padre hace el trabajo sucio y el hijo añade detalles específicos.

Java
```
class Empleado {
    public double calcularBono() {
        return 1000.0; // Bono base para todos
    }
}

class Gerente extends Empleado {
    @Override
    public double calcularBono() {
        // 1. Obtenemos la lógica base del padre
        double bonoBase = super.calcularBono();

        // 2. Añadimos la lógica específica del gerente
        return bonoBase + 500.0;
    }
}
```

C. Las 3 Reglas de Oro de la Sobreescritura

Para que la sobreescritura sea válida, debes respetar este contrato estrictamente:

1. **Firma del Método:** El nombre del método y la lista de parámetros deben ser **idénticos**.
2. **Visibilidad (Access Modifiers):** No puedes hacer el método *más privado* que en el padre.

   - Si el padre es `public`, el hijo debe ser `public`.
   - Si el padre es `protected`, el hijo puede ser `protected` o `public` (pero no `private`).
   - *Razón:* Si un usuario espera poder llamar al método en el padre, debe poder hacerlo también en el hijo (Principio de Sustitución).

3. Tipo de Retorno: Debe ser el mismo tipo o un subtipo (covariante).

   - Si el padre devuelve `Number`, el hijo puede devolver `Integer`.

Importante: Overriding vs Overloading. No confundas **Sobreescritura (Overriding)** con **Sobrecarga (Overloading).**

- **Overriding:** Mismo nombre, mismos parámetros (Herencia). Cambia el comportamiento.
- **Overloading:** Mismo nombre, *diferentes* parámetros (En la misma clase). Es solo conveniencia para usar el mismo nombre.

2.3. Restricciones: Clases y métodos `final` (Inmutabilidad y prohibición de herencia)

La palabra clave `final` en Java es el equivalente a poner un candado. Dependiendo de dónde lo coloques, bloqueas una capacidad diferente del lenguaje.

A. Métodos `final`: "Puedes usarlo, pero no cambiarlo"

Cuando marcas un método como `final`, estás permitiendo que las clases hijas lo hereden y lo utilicen, pero les **prohíbes terminantemente sobreescribirlo** (`@Override`).

¿Por qué hacerlo? Para garantizar que un algoritmo crítico se ejecute siempre de la misma manera, sin importar qué subclase se esté utilizando. Es vital para la seguridad y la consistencia de las reglas de negocio.

Ejemplo: Un sistema de pago
Java
```
class ProcesadorPago {
    // Este método puede ser modificado por los hijos (ej. Paypal
vs Tarjeta)
    public void conectarBanco() {
        System.out.println("Conectando...");
    }

    // FINAL: Nadie debe poder alterar la lógica de auditoría.
    // Es una regla de seguridad inquebrantable.
    public final void registrarAuditoria(String usuario) {
        System.out.println("Guardando log seguro para: " +
usuario);
        // Lógica compleja de encriptación que no queremos que un
hijo rompa
    }
}

class PagoBitcoin extends ProcesadorPago {
    @Override
    public void conectarBanco() { ... } // OK

    // @Override
    // public void registrarAuditoria(String u) { ... }
    // ❌ ERROR DE COMPILACIÓN: "Cannot override the final method"
}
```

> **Nota técnica:** Los métodos `private` son implícitamente `final`, porque si una subclase no puede ver el método, tampoco puede sobreescribirlo.

B. Clases `final`: "El fin del linaje"

Cuando declaras una clase entera como `final`, estás **prohibiendo la herencia**. Ninguna otra clase puede usar `extends` con ella.

- **¿Por qué hacerlo?**

1. **Seguridad:** Si creas una clase perfecta y segura, no quieres que alguien cree una subclase maliciosa que se haga pasar por ella.
2. **Diseño:** La clase no fue diseñada para ser extendida y hacerlo podría romper su funcionamiento interno.

El ejemplo clásico: `String` La clase `String` en Java es `final`. ¿Por qué? Imagina que pudieras hacer: `class MiStringMalicioso extends String`. Podrías cambiar cómo funcionan las contraseñas o las rutas de archivos en todo el sistema. Al ser final, Java garantiza que un String siempre se comporte como un String.

Java
```
// Clase marcada como final
public final class SistemaSeguridad {
    public boolean validarAcceso(String key) {
        return "1234".equals(key);
    }
}

// ❌ ERROR DE COMPILACIÓN: "Cannot inherit from final
SistemaSeguridad"
// class HackerSystem extends SistemaSeguridad { }
```

C. El concepto de Inmutabilidad

El modificador `final` es el ingrediente principal para crear **Objetos Inmutables**.

Un objeto inmutable es aquel cuyo estado (sus datos) **no puede cambiar** una vez que ha sido creado. Si quieres cambiar algo, tienes que crear un objeto nuevo.

Receta para una Clase Inmutable:

1. Declarar la clase como `final` (para que nadie la extienda y añada campos mutables).
2. Declarar todos los atributos como `private` y `final` (se asignan una vez en el constructor y nunca más cambian).
3. **No** incluir métodos "setters" (como `setNombre`).

Ejemplo de diseño Inmutable:
Java
```
public final class Moneda {
    private final String codigo; // ej. "USD"
    private final double valor;

    public Moneda(String codigo, double valor) {
        this.codigo = codigo;
        this.valor = valor;
    }

    // Solo Getters, nada de Setters
    public String getCodigo() { return codigo; }
    public double getValor() { return valor; }

    // Si queremos cambiar el valor, devolvemos un NUEVO objeto
    public Moneda convertir(double tasa) {
        return new Moneda(this.codigo, this.valor * tasa);
```

```
    }
}
```

Ventajas de la inmutabilidad:

- Son intrínsecamente seguros en entornos multi-hilo (Thread-Safe).
- Son fáciles de entender y testear (no tienen "efectos secundarios" ocultos).

Resumen Visual de Restricciones

Modificador	Aplicado a Método	Aplicado a Clase
Sin `final`	Se puede sobreescribir (`@Override`).	Se puede heredar (`extends`).
Con `final`	**Bloqueado:** Ejecución fija.	**Bloqueado:** No puede tener hijos.

2.4. Abstracción: Clases y métodos `abstract` (Plantillas incompletas)

Imagina que eres un arquitecto. Creas un plano para una cadena de restaurantes. Sabes dónde va la cocina y los baños (eso es fijo), pero dejas un espacio en blanco en el plano para la "Decoración del Salón", porque eso depende de si el restaurante está en la playa o en la ciudad.

En Java, eso es una **Clase Abstracta**.

A. Clases Abstractas: El Concepto General

Una clase declarada con `abstract` representa un concepto general que no está listo para ser usado por sí solo.

- **Regla de Hierro:** NO puedes crear objetos de una clase abstracta. `Animal a = new Animal();` // ERROR: Animal es abstracto, ¿qué animal es?
- **Propósito:** Sirve exclusivamente como padre (superclase) para agrupar atributos y comportamientos comunes.

B. Métodos Abstractos: El Contrato de "Rellene aquí"

Un método `abstract` se define en la clase padre pero **no tiene cuerpo** (no tiene llaves `{}`). Termina en punto y coma.

- **El Mandato:** Al declarar un método abstracto, le estás diciendo al compilador: *"Cualquier clase hija normal (concreta) QUE HEREDE de mí, ESTÁ OBLIGADA a escribir el código de este método"*.

Ejemplo Práctico: Sistema de Exportación de Archivos

Imagina un sistema que exporta documentos. Todos los documentos tienen un nombre y una fecha (código compartido), pero la forma de generar el archivo cambia si es PDF o Excel (código abstracto).

Java
```
// 1. La clase es abstracta porque contiene métodos abstractos
public abstract class Exportador {
    private String nombreArchivo;

    public Exportador(String nombre) {
        this.nombreArchivo = nombre;
    }

    // MÉTODO CONCRETO:
    // Lógica compartida. Todos los hijos la usan igual.
    public void firmarDigitalmente() {
        System.out.println("Añadiendo firma digital a " +
nombreArchivo);
    }

    // MÉTODO ABSTRACTO:
    // "Yo sé que hay que exportar, pero no sé cómo hacerlo. Hija,
encárgate tú."
    public abstract void generarArchivo();
}
```

Ahora, creamos las implementaciones concretas:

Java
```
// Clase Hija 1: PDF
public class ExportadorPDF extends Exportador {
    public ExportadorPDF(String nombre) { super(nombre); }

    @Override
    public void generarArchivo() {
        // Implementación obligatoria específica para PDF
        System.out.println("Generando estructura PDF y
comprimiendo...");
    }
}

// Clase Hija 2: CSV
public class ExportadorCSV extends Exportador {
    public ExportadorCSV(String nombre) { super(nombre); }

    @Override
    public void generarArchivo() {
        // Implementación obligatoria específica para CSV
        System.out.println("Escribiendo valores separados por
comas...");
    }
}
```

C. ¿Por qué usar `abstract` en el diseño?

1. **Evitar Instancias Absurdas:** Evitas que alguien haga `new Exportador()`. Un exportador genérico no sirve para nada; necesitas uno específico.
2. **Polimorfismo Garantizado:** Puedes tener una lista de `Exportador` y llamar a `generarArchivo()` en todos ellos. Java sabrá ejecutar la versión correcta (PDF o CSV) de cada uno.
3. **Código DRY (Don't Repeat Yourself):** Escribes el código común (como `firmarDigitalmente` o el manejo del nombre) una sola vez en el padre, y obligas a personalizar solo lo necesario.

Reglas Técnicas

- Si una clase tiene **al menos un** método abstracto, la clase **debe** ser declarada `abstract`.
- Una clase abstracta **puede** tener métodos normales (con código) y variables de estado.
- Si una subclase hereda de una abstracta y no implementa *todos* los métodos abstractos, esa subclase también debe ser abstracta.

3. Interfaces

3.1. Concepto y Definición: Contratos de comportamiento

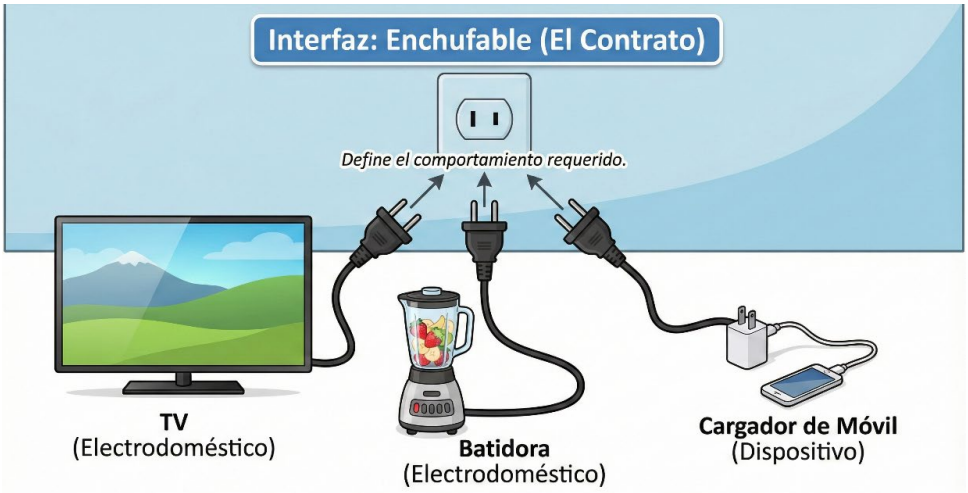
Hasta ahora, con `extends`, hemos definido qué es un objeto (un Perro es un Animal). Con las `interfaces`, definimos qué puede hacer un objeto, sin importar su linaje o familia.

Una **Interfaz** es una colección de métodos abstractos (y constantes) que definen un **contrato de comportamiento**.

A. La Metáfora del "Enchufe Universal"

Piensa en un enchufe de pared estándar.

- **El contrato (Interfaz):** "Si tienes dos patas metálicas separadas por X distancia y aguantas 220V, te daré electricidad".
- **Los implementadores:** No importa si conectas una **Tv**, una **Batidora** o un **Cargador de Móvil**. Son objetos totalmente distintos, con padres distintos (`Electrodoméstico`, `Dispositivo`), pero todos cumplen el contrato del enchufe.



Objetos diferentes (Clases) implementando la misma Interfaz.

B. Sintaxis Básica

En lugar de `class`, usamos `interface`. En lugar de `extends`, usamos `implements`.

1. Definición del Contrato. En una interfaz, no escribimos cómo funcionan las cosas, solo enumeramos qué capacidades se requieren.

Java

```
// Nombramiento: Suele ser un sustantivo o adjetivo (ej.
"Imprimible", "Auditable")
public interface Imprimible {
    // Los métodos son IMPLÍCITAMENTE "public abstract"
    // No hace falta escribir esas palabras, Java lo asume.
    void imprimir();

    void obtenerDetalles();
}
```

2. Firma del Contrato (Implementación). Cualquier clase que use implements firma el contrato. Esto significa que está obligada a escribir el código de todos los métodos de la interfaz (o ser declarada abstracta).

Java

```
// Clase 1: Un Documento (Familia Archivos)
public class InformePDF extends Archivo implements Imprimible {
    @Override
    public void imprimir() {
        System.out.println("Enviando bytes al buffer de la
impresora...");
    }

    @Override
    public void obtenerDetalles() {
        System.out.println("Informe PDF - 2MB");
    }
}

// Clase 2: Una Foto (Familia Multimedia)
// ¡No tiene NADA que ver con un Informe, pero ambos son
Imprimibles!
public class FotoFamiliar extends Imagen implements Imprimible {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo en papel fotográfico
brillante...");
    }

    @Override
    public void obtenerDetalles() {
        System.out.println("Foto JPEG - 1080p");
    }
}
```

C. Características Clave

1. **Desacoplamiento total:** Una interfaz permite conectar clases que no tienen relación jerárquica. Esto reduce la dependencia entre partes de tu código (Acoplamiento débil).

2. **La "Herencia Múltiple" de Java:** Java no permite heredar de dos padres (`extends A, B` es ilegal). Pero sí permite implementar múltiples interfaces.

Java
```
// Un Smartphone es un Teléfono, pero también es una Cámara y un
Reproductor de Música.
public class Smartphone extends Telefono implements Camara,
ReproductorMusica {
    // Debe implementar los métodos de Camara Y de
ReproductorMusica
}
```

3. **Polimorfismo puro:** Puedes tratar a objetos muy distintos bajo el mismo tipo de interfaz.

Java
```
Imprimible a = new InformePDF();
Imprimible b = new FotoFamiliar();

// Lista de cosas imprimibles, sin importar qué son realmente
List<Imprimible> colaImpresion = new ArrayList<>();
colaImpresion.add(a);
colaImpresion.add(b);

for(Imprimible item : colaImpresion) {
    item.imprimir(); // Funciona para ambos
}
```

D. Abstract vs Interface: La distinción rápida

Es común confundirse, así que aquí tienes la regla para decidir:

- Usa **Clase Abstracta** cuando la relación es **"ES-UN"** (Is-A). Comparten código y atributos. Ej: `Perro` es un `Animal`.
- Usa **Interfaz** cuando la relación es **"PUEDE-HACER"** (Can-Do) o **"SE-COMPORTA-COMO"**. Es una capacidad añadida. Ej: `Perro` puede ser `Entrenable` (Interfaz).

3.2. Interfaces vs Clases Abstractas: Cuándo usar cada una y diferencias clave

Para dominar esto, hay que entender que luchan en dos frentes diferentes: **Relación** (Diseño) y **Capacidad** (Técnica).

A. Diferencias Técnicas (La Tabla de la Verdad)

Aquí tienes los límites rígidos que impone el lenguaje:

Característica	Clase Abstracta	Interfaz
Herencia	Una clase solo puede extender **una** clase abstracta (Herencia simple).	Una clase puede implementar **múltiples** interfaces.
Estado (Variables)	Puede tener variables de instancia (estado) mutables (no finales).	Solo puede tener constantes (`public static final`). No guarda estado.
Constructores	**Sí** tiene constructores (para inicializar su estado).	**No** tiene constructores.
Métodos	Puede tener métodos `private`, `protected`, `public` y `static`.	Por defecto son `public`. (Desde Java 9 permite privados, pero es avanzado).
Velocidad	Ligeramente más rápida (enlace directo).	Ligeramente más lenta (búsqueda de implementación), aunque hoy día es imperceptible.

B. Diferencias de Diseño (La Filosofía)

Aquí es donde decides como arquitecto de software.

1. Clase Abstracta: La relación "ES-UN" (Is-A). Úsala cuando quieras compartir código y estado entre clases estrechamente relacionadas. Estás definiendo el núcleo de identidad de un objeto.

- *Ejemplo:* `Humano` y `Perro`. Ambos respiran, tienen edad, tienen nombre.
- *Diseño:* `Clase Abstracta SerVivo`.
    - Tiene variable `int edad`.
    - Tiene método real `envejecer()`.
    - Tiene método abstracto `sonido()`.

2. Interfaz: La relación "PUEDE-HACER" (Can-Do) Úsala para definir una capacidad o rol que pueden tener objetos que no tienen nada que ver entre sí. Estás definiendo un periférico o habilidad.

- *Ejemplo:* `Perro` y `Dron`.
- Un `Perro` es un animal. Un `Dron` es una máquina. No comparten ancestros.
- Pero ambos **pueden ser rastreados** por GPS.
- *Diseño:* `Interfaz Rastreable`.
    - Método `obtenerCoordenadas()`.

C. Ejemplo Comparativo: El problema del "Robot-Mascota"

Imagina que usamos Clases Abstractas para todo.

Java

```
abstract class Animal {
    abstract void comer();
}

abstract class Maquina {
    abstract void recargarBateria();
}

// Interfaz para la capacidad de "Jugar"
interface Jugable {
    void jugar();
}
```

Ahora mira cómo aplicamos esto a tres entidades distintas:
Java

```
// 1. PERRO REAL: Es un Animal, y se puede jugar con él.
class Perro extends Animal implements Jugable {
    @Override void comer() { ... }
    @Override public void jugar() { ... }
}

// 2. COCHE TELEDIRIGIDO: Es una Máquina, y se puede jugar con él.
class CocheJuguete extends Maquina implements Jugable {
    @Override void recargarBateria() { ... }
    @Override public void jugar() { ... }
}

// 3. LOBO: Es un Animal, pero NO es jugable (te muerde).
class Lobo extends Animal {
    @Override void comer() { ... }
    // No implementa Jugable
}
```

Análisis:

- Si `Jugable` fuera una clase abstracta, `CocheJuguete` no podría heredar de ella porque ya hereda de `Maquina`. **Aquí gana la Interfaz.**
- Si `Animal` fuera una interfaz, no podríamos guardar la variable `nivelHambre` común para todos los animales dentro de ella. **Aquí gana la Clase Abstracta.**

D. Guía de Decisión Rápida

Usa **Clase Abstracta** si:

- ✅ Necesitas compartir código entre varias clases estrechamente relacionadas.
- ✅ Necesitas que las clases hijas hereden variables (estado) que no sean constantes.
- ✅ Quieres declarar campos o métodos `non-public` (protected/private).

Usa **Interfaz** si:

- ✅ Esperas que clases no relacionadas implementen tu funcionalidad.

- ✅ Quieres especificar el comportamiento de un tipo de dato, pero no te importa quién lo implemente.
- ✅ Necesitas aprovechar la herencia múltiple de tipo (un objeto puede ser `Serializable`, `Clonable` y `Ejecutable` a la vez).

4. Polimorfismo

4.1. Concepto General: Una interfaz, múltiples formas

La palabra proviene del griego: *Poly* (muchos) y *Morphe* (formas).

En Java, el polimorfismo es la capacidad de un objeto de **comportarse de diferentes maneras** dependiendo de su implementación específica, aunque lo estemos tratando de forma genérica.

> **El Mantra del Polimorfismo:** "No me importa qué eres exactamente, solo me importa lo que puedes hacer".

A. La Metáfora del Mando a Distancia (Control Remoto)

Imagina que tienes un **mando universal** (la referencia genérica) con un botón "Encender" (el método).

1. Apuntas a la **TV** y pulsas "Encender" -> La pantalla se ilumina.
2. Apuntas al **Aire Acondicionado** y pulsas "Encender" -> Empieza a salir aire frío.
3. Apuntas al **Equipo de Música** y pulsas "Encender" -> Suena una canción.

Tú (el programador) has hecho la **misma acción** (pulsar botón "Encender" en el mando), pero el **resultado** ha sido diferente según el objeto al que apuntabas. Eso es polimorfismo.

B. En Código: Referencias Genéricas vs Objetos Específicos

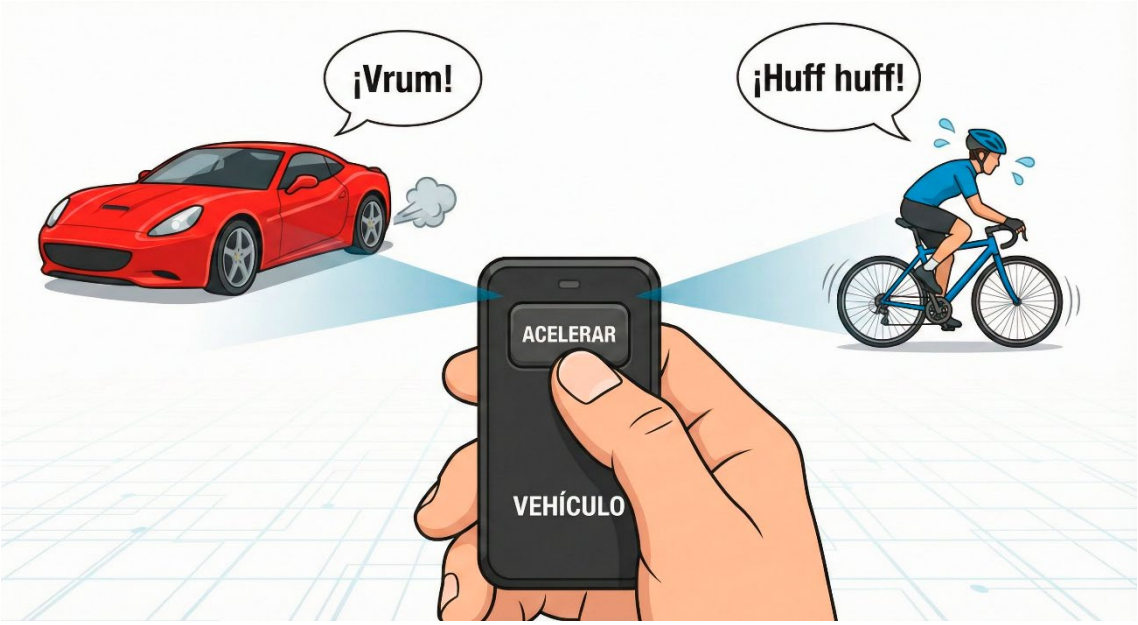
La magia técnica ocurre cuando usamos una variable de tipo **Padre (o Interfaz)** para guardar un objeto de tipo **Hijo**.

Java
```
// Definimos el contrato general
abstract class Vehiculo {
    abstract void acelerar();
}

class Coche extends Vehiculo {
    @Override void acelerar() {
        System.out.println("El coche acelera quemando gasolina...
¡Vrum!");
    }
}

class Bicicleta extends Vehiculo {
    @Override void acelerar() {
        System.out.println("La bici acelera pedaleando fuerte...
¡Huff huff!");
    }
```

```
}
```

Ahora, observa el polimorfismo en acción:
Java

```
public class Main {
    public static void main(String[] args) {
        // 1. Declaramos la variable del tipo GENERAL (Padre)
        Vehiculo miVehiculo;

        // 2. Le asignamos una forma específica (Coche)
        miVehiculo = new Coche();
        miVehiculo.acelerar(); // Salida: "El coche acelera..."

        // 3. ¡Magia! Usamos la MISMA variable para otra forma
(Bicicleta)
        miVehiculo = new Bicicleta();
        miVehiculo.acelerar(); // Salida: "La bici acelera..."
    }
}
```



C. ¿Por qué es esto tan importante?

Imagina que estás programando un videojuego con 100 tipos de enemigos diferentes. Sin polimorfismo, tendrías que gestionar cada uno por separado:

SIN Polimorfismo (Código rígido y difícil de mantener):
Java

```
// Tendrías que saber exactamente qué tipo es cada vez
if (enemigo es Orco) {
    orco.atacarConHacha();
} else if (enemigo es Dragon) {
    dragon.lanzarFuego();
} else if (enemigo es Mago) {
    mago.lanzarHechizo();
```

```
}
// Si añades un nuevo enemigo, ¡tienes que cambiar este código!
```

CON Polimorfismo (Código flexible y escalable):
Java

```
// Solo necesitas saber que es un Enemigo y que puede atacar
for (Enemigo e : listaDeEnemigos) {
    e.atacar();
    // Java decide automáticamente si lanza fuego, usa hacha o
hechizo
    // dependiendo de qué sea 'e' en ese momento.
}
```

D. Tipos de Polimorfismo

Aunque nos centraremos en el que acabamos de ver, es útil saber que existen dos variantes principales:

1. **Polimorfismo en tiempo de compilación (Estático):** Es la **Sobrecarga (Overloading)** de métodos (mismo nombre, diferentes parámetros). El compilador sabe qué método llamar antes de ejecutar el programa.

2. **Polimorfismo en tiempo de ejecución (Dinámico):** Es la **Sobreescritura (Overriding)** que estamos viendo. Java decide mientras el programa corre (en tiempo real) a qué versión del método llamar, basándose en el objeto real que hay en memoria (`new Coche()` o `new Bici()`).

4.2. Polimorfismo en Compilación: Sobrecarga de métodos (Overloading)

La **Sobrecarga (Overloading)** ocurre cuando tienes **varios métodos con el mismo nombre** en la misma clase, pero con **diferentes listas de parámetros**.

Es la forma en que Java te dice: *"No necesitas inventar nombres raros como `sumarDosEnteros` y `sumarDosDecimales`. Llámalos a todos `sumar` y yo sabré cuál usar basándome en lo que me entregues"*.

A. La Regla de Oro: La Firma del Método

Para que la sobrecarga funcione, la **firma** debe cambiar. La firma se compone de:

1. Nombre del método.
2. Tipos de parámetros.
3. Orden de los parámetros.

Importante: El **tipo de retorno** NO forma parte de la firma para la sobrecarga. No puedes tener dos métodos que solo se diferencien en lo que devuelven.

Ejemplo de Sobrecarga Válida:
Java

```java
public class Calculadora {
    // 1. Sumar dos enteros
    public int sumar(int a, int b) {
        return a + b;
    }

    // 2. Sumar tres enteros (Cambia el NÚMERO de parámetros)
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }

    // 3. Sumar decimales (Cambia el TIPO de parámetros)
    public double sumar(double a, double b) {
        return a + b;
    }
}
```

Uso:

Java

```java
Calculadora calc = new Calculadora();
calc.sumar(5, 10);      // El compilador sabe que debe usar la
versión 1
calc.sumar(5.5, 2.3);   // El compilador sabe que debe usar la
versión 3
```

B. Sobrecarga de Constructores

Este es el uso más común. A menudo quieres ofrecer diferentes formas de crear un objeto: una forma rápida (por defecto) y una forma detallada.

Java

```java
public class Usuario {
    private String nombre;
    private String rol;

    // Constructor 1: Solo con nombre (Rol por defecto)
    public Usuario(String nombre) {
        this.nombre = nombre;
        this.rol = "INVITADO";
    }

    // Constructor 2: Con nombre y rol específico
    public Usuario(String nombre, String rol) {
        this.nombre = nombre;
        this.rol = rol;
    }

    // Constructor 3: Vacío (Valores por defecto)
    public Usuario() {
        this.nombre = "Anónimo";
        this.rol = "INVITADO";
    }
}
```

C. Diferencias Clave: Overloading vs Overriding

Esta tabla te salvará en muchas confusiones futuras:

Característica	Sobrecarga (Overloading)	Sobreescritura (Overriding)
Objetivo	Aumentar la legibilidad (mismo nombre para acciones similares).	Cambiar el comportamiento de un método heredado.
Ubicación	Ocurre dentro de la **misma clase**.	Ocurre entre **Padre** e **Hijo** (Herencia).
Parámetros	**DEBEN ser diferentes**.	**DEBEN ser idénticos**.
Retorno	Puede cambiar libremente.	Debe ser el mismo (o compatible/covariante).
Momento	**Compilación** (Static Binding). El compilador elige cuál usar al leer el código.	**Ejecución** (Dynamic Binding). La JVM elige cuál usar según el objeto real.

D. Un caso especial: Varargs (Argumentos variables)

Java permite una forma "suprema" de sobrecarga usando `...`. Esto permite pasar un número indeterminado de argumentos del mismo tipo.
Java

```
// Recibe desde 0 hasta N strings
public void imprimirNombres(String... nombres) {
    for (String n : nombres) {
        System.out.println(n);
    }
}

// Uso:
imprimirNombres("Ana");
imprimirNombres("Ana", "Luis", "Pedro");
imprimirNombres(); // Incluso vacío funciona
```

4.3. Polimorfismo en Ejecución: Ligadura dinámica (Dynamic Binding)

La **Ligadura Dinámica** (o *Late Binding*) es el proceso mediante el cual Java decide qué implementación específica de un método debe ejecutar **mientras el programa está corriendo**, basándose en el objeto real que existe en la memoria, no en el tipo de la variable que lo contiene.

A. El concepto: "La Apariencia vs. La Realidad"

Para entender esto, debemos distinguir dos cosas que ocurren en tu código:

1. **Tipo de Referencia (Declarado):** Es lo que el compilador ve. Determina **qué** métodos puedes llamar.

    - `Animal a;` -> El compilador solo sabe que `a` es un Animal.

2. **Tipo de Objeto (Instancia Real):** Es lo que se crea en la memoria (Heap). Determina **cómo** se comportan esos métodos.

    - `a = new Perro();` -> En memoria, hay un Perro.

La Regla de Oro: En Java, las llamadas a métodos de instancia son **polimórficas**. La JVM busca el método en el objeto real (el `new ...`), no en el tipo de la variable.

B. Ejemplo Práctico: El "Engaño" al Compilador

Imagina este escenario clásico:

Java
```
class Instrumento {
    void tocar() { System.out.println("Sonido genérico..."); }
}

class Guitarra extends Instrumento {
    @Override // Sobreescritura
    void tocar() { System.out.println("¡Acorde de Guitarra!"); }
}

class Violin extends Instrumento {
    @Override // Sobreescritura
    void tocar() { System.out.println("Melodía de Violín..."); }
}
```

Ahora mira la ejecución paso a paso:

Java
```
public class Concierto {
    public static void main(String[] args) {
        // 1. COMPILACIÓN:
        // Java ve una variable de tipo 'Instrumento'.
        // El compilador verifica: "¿Tiene Instrumento el método
tocar()?"
        // Sí. Compilación exitosa.
        Instrumento miInstrumento = new Guitarra();

        // 2. EJECUCIÓN (Ligadura Dinámica):
        // La JVM llega a esta línea. Mira dentro de la memoria
Heap.
        // Ve que el objeto real es una 'Guitarra'.
        // Ignora el método de Instrumento y ejecuta el de
Guitarra.
        miInstrumento.tocar();
    }
}
// Salida: "¡Acorde de Guitarra!"
```

C. La Gran Excepción: Atributos (Variables) vs Métodos

Aquí es donde el 90% de los estudiantes fallan en los exámenes. **La ligadura dinámica SOLO aplica a los métodos de instancia.**

- **Métodos:** Se resuelven en tiempo de ejecución (Gana el objeto real / Hijo).
- **Variables (Atributos):** Se resuelven en tiempo de compilación (Gana la referencia / Padre).
- **Métodos Estáticos:** Se resuelven en tiempo de compilación (Gana la referencia / Padre).

El caso trampa:

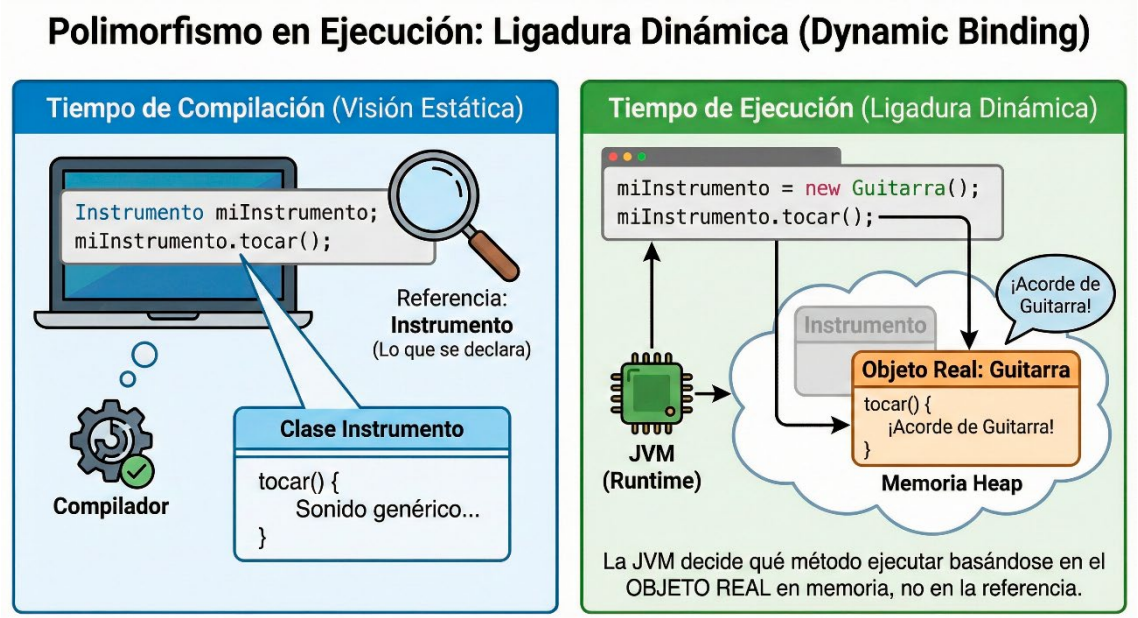
Java

```java
class Padre {
    String nombre = "Padre";
    void saludar() { System.out.println("Hola soy Padre"); }
}

class Hijo extends Padre {
    String nombre = "Hijo"; // Ocultamiento de variable (Shadowing)
    @Override
    void saludar() { System.out.println("Hola soy Hijo"); }
}

public class Prueba {
    public static void main(String[] args) {
        Padre p = new Hijo(); // Referencia Padre, Objeto Hijo

        // MÉTODO: Dinámico (Busca en el objeto real -> Hijo)
        p.saludar(); // Imprime: "Hola soy Hijo"

        // VARIABLE: Estático (Busca en la referencia -> Padre)
        System.out.println(p.nombre); // Imprime: "Padre"
    }
}
```

Consejo de diseño: Por esta razón (y por encapsulamiento), **nunca** debes acceder a las variables directamente (`p.nombre`). Usa siempre métodos getters (`p.getNombre()`). Si usas un getter, al ser un método, se aplicará el polimorfismo y obtendrás el valor correcto del hijo.

D. Resumen Visual del Binding

Tipo de Miembro	Tipo de Ligadura (Binding)	¿Quién decide?	¿En qué se fija?
Métodos (`static`)	Estática (Early Binding)	Compilador	Tipo de la Variable (Izquierda del =)
Variables	Estática (Early Binding)	Compilador	Tipo de la Variable (Izquierda del =)
Métodos (Instancia)	**Dinámica (Late Binding)**	**JVM**	**Tipo del Objeto (Derecha del =)**



4.4. Comprobación de tipos: Estática (compilador) vs Dinámica (`instanceof`)

Para entender esto, imagina que tienes una caja cerrada etiquetada como "Fruta" (Referencia). Dentro podría haber una Manzana, una Pera o un Plátano (Objeto real).

A. Comprobación Estática (El trabajo del Compilador)

El compilador de Java es conservador y pesimista. Solo confía en la **etiqueta de la caja** (el Tipo de la Referencia), nunca mira dentro.

Si tienes una variable declarada como `Animal`, el compilador solo te dejará llamar a métodos que existan en la clase `Animal`, incluso si tú sabes perfectamente que dentro hay un `Perro`.

El "Muro" de la comprobación estática:

Java

```
// 1. Referencia de tipo Animal, Objeto de tipo Perro
Animal miMascota = new Perro();

// 2. Método común (DEFINIDO EN ANIMAL)
miMascota.comer(); // ✅ OK: El compilador ve 'comer' en la clase
Animal.

// 3. Método específico (SOLO EN PERRO)
// miMascota.ladrar();
// ❌ ERROR DE COMPILACIÓN: "Cannot find symbol method ladrar()"
```

¿Por qué falla? El compilador razona así: "Tú me has dicho que miMascota es un Animal. No todos los animales ladran (los peces no lo hacen). Por seguridad, te prohíbo llamar a ladrar aquí".

B. Comprobación Dinámica (El operador `instanceof`)

Para superar la limitación del compilador, necesitamos "abrir la caja" en tiempo de ejecución y ver qué hay realmente dentro. Para eso usamos el operador `instanceof`.

`instanceof` es una pregunta binaria (true/false) que se hace a un objeto en memoria: *"¿Eres tú, o eres compatible con, el tipo X?"*

Sintaxis:
Java
```
variable instanceof Clase
```

Ejemplo de uso seguro: Siguiendo el ejemplo anterior, queremos que nuestra mascota ladre, pero solo si realmente es un perro.

Java
```
Animal miMascota = new Perro();

// Preguntamos dinámicamente: ¿Qué eres realmente?
if (miMascota instanceof Perro) {
    System.out.println("¡Es un perro! Puedo tratarlo como tal.");
    // Aquí es seguro hacer el Casting (que veremos en el 6.4.5)
} else {
    System.out.println("No es un perro, mejor no intento que
ladre.");
}
```

C. Matices importantes de `instanceof`

1. **Jerarquía de Herencia:** `instanceof` devuelve true si el objeto es de esa clase exacta O de cualquier clase hija.

Java
```
Perro firulais = new Perro();
System.out.println(firulais instanceof Perro);  // true
System.out.println(firulais instanceof Animal); // true (Un perro
ES UN animal)
System.out.println(firulais instanceof Object); // true (Todo es un
objeto)
```

2. **Seguridad con Nulos:** Si la variable es `null`, `instanceof` siempre devuelve `false`. No lanza error (NullPointerException), lo cual es muy útil para validaciones.

Java
```
Animal a = null;
boolean esPerro = a instanceof Perro; // false (y no crashea)
```

D. Modernización: Pattern Matching for `instanceof` (Java 14+)

Antes de Java 14, usar `instanceof` era verboso porque tenías que comprobar y luego convertir (cast) manualmente. Java moderno introdujo una sintaxis más limpia.

Estilo Antiguo (Pre-Java 14):
Java
```
if (obj instanceof String) {
    String s = (String) obj; // <--- Paso extra repetitivo (Casting
explícito)
    System.out.println(s.length());
```

```
}
```

Estilo Moderno (Java 14+):
Java
```
// Comprueba, convierte y asigna a la variable 's' en un solo paso
if (obj instanceof String s) {
    System.out.println(s.length()); // 's' ya es un String usable
aquí
}
```

Resumen Comparativo

Tipo de Comprobación	Quién la hace	En qué se basa	Objetivo
Estática	Compilador (Javac)	Tipo de la **Variable** (Referencia)	Garantizar que los métodos llamados existen en el contrato.
Dinámica	JVM (Runtime)	Tipo del **Objeto** (Memoria Heap)	Verificar la identidad real del objeto antes de realizar acciones específicas (como un casting).

5. Tipado Avanzado y Genéricos

5.1. Conversiones de Objetos (Casting)

En Java, el **Casting** de objetos no transforma el objeto en sí (el bloque de memoria en el Heap no cambia). Lo que cambia es la **etiqueta** (la referencia) con la que apuntamos a ese objeto. Es como cambiarte las gafas: el objeto sigue siendo el mismo, pero tú lo ves con más o menos detalles.

Existen dos direcciones en las que puedes moverte dentro de una jerarquía de herencia:

A. Upcasting (Hacia Arriba): Implícito y Seguro

El Upcasting consiste en tratar a un objeto de una clase hija como si fuera de su clase padre.

- **¿Por qué es seguro?** Porque la herencia garantiza que un Hijo **siempre** tiene todo lo que tiene el Padre. Un `Perro` siempre *es un* `Animal`. Nunca le faltarán patas ni métodos heredados.
- **Sintaxis:** Es **implícito**. No necesitas escribir código especial. Java lo hace automáticamente.

Ejemplo:
Java
```
// Creamos un objeto Perro
Perro miPerro = new Perro();

// UPCASTING: Lo guardamos en una variable de tipo Animal (Padre)
```

```
// Java dice: "Sin problema, todo perro es un animal".
Animal animalGenerico = miPerro;
```

La Consecuencia (Restricción de Visión): Al hacer upcasting, pierdes acceso a los métodos específicos del hijo. Aunque el objeto en memoria sea un `Perro`, si lo manejas con la etiqueta `Animal`, solo puedes pedirle cosas de `Animal`.

Java
```
animalGenerico.comer();   // ✅ OK: Definido en Animal
// animalGenerico.ladrar(); // ❌ ERROR DE COMPILACIÓN: Animal no
sabe ladrar
```

B. Downcasting (Hacia Abajo): Explícito y Riesgoso

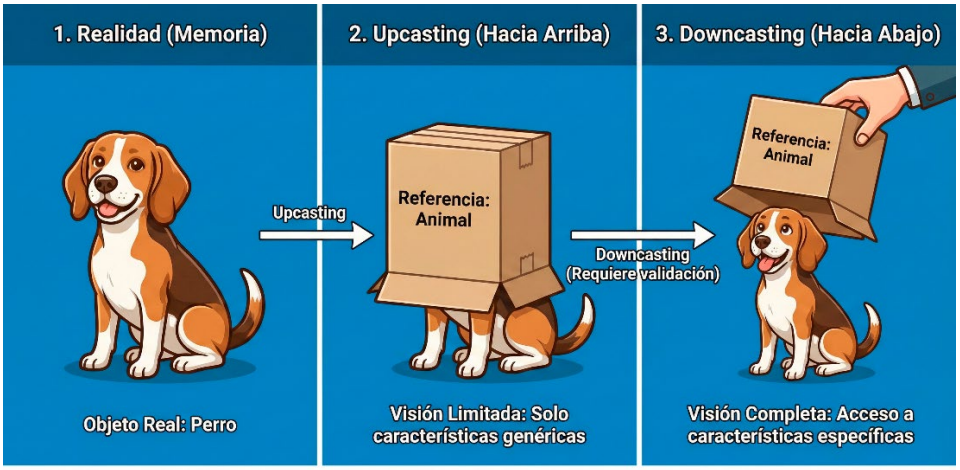
El Downcasting consiste en recuperar la forma específica de un objeto que teníamos guardado en una referencia genérica (Padre -> Hijo).

- **¿Por qué es riesgoso?** Porque una variable `Animal` podría estar apuntando a un `Gato`. Si intentas forzar a ese `Gato` a ser un `Perro`, el programa fallará.
- **Sintaxis:** Es **explícito**. Debes usar paréntesis `(TipoHijo)` para "firmar el contrato de responsabilidad" ante el compilador.

Ejemplo Correcto (Con validación previa):
Java
```
Animal miMascota = new Perro(); // Upcasting inicial

// Queremos que ladre. Necesitamos recuperar la referencia de
Perro.
if (miMascota instanceof Perro) {
    // DOWNCASTING: (Perro) variable
    Perro recuperado = (Perro) miMascota;
    recuperado.ladrar(); // ✅ ¡Ahora sí podemos acceder a métodos
específicos!
}
```

Ejemplo del Desastre (ClassCastException): Imagina que mentimos al compilador.

Java
```
Animal miMascota = new Gato(); // Es un gato real

// El compilador confía en ti porque usas el casting explícito
(Gato -> Perro??)
// Sintácticamente "Perro p = (Perro) animal" es válido para el
compilador.
Perro perroFalso = (Perro) miMascota;

// 💥 ERROR EN EJECUCIÓN: ClassCastException
// Java detecta en memoria que intentas meter un Gato en una
variable Perro.
```



La "Caja" de la Referencia: Cambia cómo vemos el objeto, no el objeto mismo.

C. Resumen de Seguridad

Tipo	Dirección	Sintaxis	Seguridad	Efecto
Upcasting	Hijo → Padre	`Padre p = hijo;`	100% Seguro	Generalización (Pierdes métodos específicos).
Downcasting	Padre → Hijo	`Hijo h = (Hijo) padre;`	**Peligroso**	Especialización (Recuperas métodos específicos). Requiere `instanceof`.

Más info: El compilador te cuida en el Upcasting. En el Downcasting, tú estás solo frente al peligro (por eso debes usar `instanceof`).

5.2. Genéricos: Introducción a las clases y tipos parametrizados (`<T>`)

Antes de Java 5, las colecciones y clases contenedoras funcionaban con `Object`. Eso significaba que podías meter una "Manzana" y una "Pera" en la misma bolsa, pero al sacarlas, tenías que adivinar qué eran y hacer un *casting*.

Los **Genéricos** te permiten especificar (parametrizar) el tipo de dato exacto que una clase o método va a utilizar. Es como ponerle una **etiqueta estricta** a una caja.

A. El problema: El "Cajón de Sastre" (Sin Genéricos)

Imagina una clase antigua que guarda un objeto. Al usar `Object`, perdemos la identidad del dato.

Java
```
// Clase sin genéricos
public class CajaAntigua {
    private Object objeto; // Acepta CUALQUIER cosa
```

```
    public void guardar(Object objeto) { this.objeto = objeto; }

    public Object sacar() { return this.objeto; }
}

// Uso peligroso:
CajaAntigua caja = new CajaAntigua();
caja.guardar("Hola Mundo"); // Guardamos un String

// PROBLEMA:
// 1. Necesito hacer Downcasting explícito (riesgoso).
// 2. Si me equivoco de tipo, explota en ejecución.
Integer numero = (Integer) caja.sacar(); // ✸ ClassCastException
en tiempo real
```

B. La Solución: Tipos Parametrizados `<T>`

Con Genéricos, definimos la clase con un "hueco" o variable de tipo, representada comúnmente por `<T>` (de *Type*).

Cuando usas la clase, **rellenas ese hueco** con el tipo que quieras. El compilador entonces garantiza que *solo* ese tipo pueda entrar o salir.

Java

```
// Clase Genérica
// <T> es un marcador de posición. "T" será lo que el usuario
decida luego.
public class Caja<T> {
    private T contenido; // La variable es de tipo T

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return this.contenido; // Devuelve T (sin necesidad de
casting)
    }
}
```

Uso Seguro (Type-Safe):

Java

```
// "Hola Java, quiero una Caja donde T sea igual a String"
Caja<String> cajaDeTexto = new Caja<>();

cajaDeTexto.guardar("Hola"); // ✅ Correcto
// cajaDeTexto.guardar(123); // ❌ ERROR DE COMPILACIÓN: Java te
avisa ANTES de ejecutar.

// ¡Magia! No hace falta casting. Java sabe que lo que sale es
String.
String texto = cajaDeTexto.sacar();
```

C. Convenciones de Nombres

Verás letras mayúsculas simples dentro de los diamantes `<  >`. Aunque puedes usar cualquier nombre, estas son las convenciones universales:

- **`E`** - Elemento (usado mucho en colecciones como `List<E>`).
- **`K`** - Key (Clave, en mapas).
- **`V`** - Value (Valor, en mapas).
- **`N`** - Number (Números).
- **`T`** - Type (Tipo genérico general).

D. Beneficios Clave

1. **Seguridad en Compilación:** Los errores de tipo se detectan mientras escribes el código (líneas rojas), no cuando el cliente usa la aplicación (crashes).

2. **Eliminación de Castings:** El código es más limpio y legible porque desaparecen los (String), (Integer), etc.

3. **Reutilización de Código:** Escribes el algoritmo una vez (ej. ordenar una lista) y funciona para Integer, String, Usuario, o cualquier clase que le pases.

Resumen Visual

Característica	Sin Genéricos (Raw Types)	Con Genéricos (<T>)
Definición	`List lista = ...`	`List<String> lista = ...`
Lo que acepta	`Object` (Todo)	Solo `String` (Estricto)
Detección de error	En Ejecución (Tarde) 💥	En Compilación (Temprano) 🛡
Obtener datos	Requiere Casting `(String)`	Directo y limpio

Resumen

1. El Control del Diseño (Modificadores)

Aprendimos que no todo debe ser público ni todo debe ser modificable. Los modificadores definen las reglas de juego:

- **`static`**: El miembro pertenece a la **clase**, no a los objetos (compartido).
- **`final`**: El "candado" de Java. Evita que las clases tengan hijos o que los métodos sean sobreescritos (fundamental para la **inmutabilidad**).
- **`protected`**: El acceso intermedio, diseñado específicamente para la **herencia**.
- **`abstract`**: Define plantillas incompletas. Obliga a las subclases a implementar la lógica, impidiendo crear objetos "genéricos" sin sentido.

2. Abstracción y Contratos (Clases vs. Interfaces)

Entendimos la diferencia filosófica entre "ser algo" y "poder hacer algo":

- **Clases Abstractas**: Representan una identidad compartida (**ES-UN**). Permiten tener estado (variables) y código base.
- **Interfaces**: Representan un contrato de comportamiento (**PUEDE-HACER**). Son la clave para el desacoplamiento y permiten la "herencia múltiple" de tipos en Java.

3. El Poder del Polimorfismo

Es la capacidad de tratar objetos distintos de manera uniforme. Se manifiesta de dos formas:

1. **Estático (Sobrecarga/Overloading)**: Mismo nombre de método, diferentes parámetros. Se decide al **compilar**.
2. **Dinámico (Ligadura Dinámica)**: Java decide en **tiempo de ejecución** qué método llamar basándose en el objeto real en memoria, no en la variable.

Clave: Gracias al polimorfismo, tu código puede interactuar con clases que aún no has escrito, siempre que cumplan con el contrato (interfaz o padre).

4. Seguridad de Tipos y Genéricos

Cerramos la unidad aprendiendo a manejar los datos con precisión quirúrgica:

- **Casting**: El arte de subir (**Upcasting**, seguro) y bajar (**Downcasting**, riesgoso) por la jerarquía.
- `instanceof`: Nuestra herramienta de inspección para evitar errores de ejecución.
- **Genéricos (`<T>`)**: La solución definitiva para crear clases flexibles (que funcionen con cualquier tipo) sin sacrificar la seguridad y evitando los peligrosos castings manuales.

Cuadro Sinóptico de Decisiones

Si necesitas...	**Usa...**
Compartir una variable entre todos los objetos	`static`
Prohibir que alguien herede de tu clase	`final`
Obligar a los hijos a implementar un método	`abstract`
Que clases sin relación compartan una habilidad	`interface`
Detectar errores de tipo antes de ejecutar	`Generics <T>`